

MergeSort(A, p, r)

if (p > r)

q = $\lfloor \frac{p+r}{2} \rfloor$

MergeSort(A, p, q)

MergeSort(A, q+1, r)

Intercala(A, p, q, r)

Intercala(A, p, q, r)

n₁ = $\frac{p+q}{2}$

n₂ = $\frac{q+r}{2}$

R[0...n₁] = A[0...n₁]

L[q+1...n₂] = A[q+1...n₂]

i = 0, j = 0, k = 0

For (i = 1, i ≤ n₁, i++)

if (R[i] ≤ L[j] and R[j] ≤ A[i-j])

if (R[i] ≤ L[j]) j++ k++

else if (R[i] > L[j])

A[k] = L[j] k++

else

A[k] = R[i] j++

return A

2, c

2) Busca(A[], p, r)

meio = $\frac{p+(r-p)}{2}$

if (A[meio] > A[meio+1] and A[meio] < A[meio-1])

return meio

else if (A[meio] > A[meio+1])

busca(A, p, meio-1)

else

busca(A, meio+1, r)

2, 0

- O algoritmo divide o array em subpartes e vai comparando o elemento do centro da subparte com seus vizinhos até encontrar o índice desejado.

3) Max-Min(A[], p, r)

MergeSort(A, p, r)

Max = A[r]

Min = A[p]

MergeSort(A[], p, r)

if (p > r)

q = $\lfloor \frac{p+r}{2} \rfloor$

MergeSort(A, p, q)

MergeSort(A, q+1, r)

Intercala(A, p, q, r)

Intercala(A, p, q, r)

n₁ = $\frac{p+q}{2}$

n₂ = $\frac{q+r}{2}$

R[0...n₁] = A[0...n₁]

L[q+1...n₂] = A[q+1...n₂]

i = j = k = 0

For (i = 1, i ≤ n₁, i++)

if (R[i] ≤ L[j])

A[k] = R[i] j++ k++

else if (R[i] > L[j])

A[k] = L[j] k++

else

A[k] = R[i] j++

return A

- Esse algoritmo inicia ordenando o array por meio do MergeSort. tendo o array ordenado, basta escolher o primeiro e o último índice para obter o mínimo e o máximo valor, respectivamente.

$O(n \log n)$

Ineficiente

X

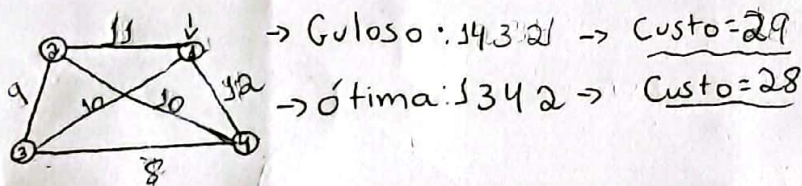
1) Guloso (L) ^{início fim lista postos}

$S = \emptyset$
 Ordene $P[i]$ em ordem decrescente em relação ao preço
 while ($\text{dist}(a, P[i]) > L$)
 $i++$
 $S = S \cup P[i]$

Guloso (L, $P[i]$, b, $P[i]$)

→ Deveria ver se chega ao final sem abastecer

5) Contra-exemplo:



Logo, o algoritmo não é exato

6) a) Guloso (I[], K)

$S = \emptyset$
 Ordene I decrescente em relação a $\left(\frac{v_i}{w_i}\right)$
 while ($w_i \leq K$)
 $S = S \cup a_i$
 $K = K - w_i$
 $i++$
 return S

→ E se não couber todo?
 (1.0)

b) (Sim) pois o algoritmo acima estará considerando os algoritmos de maior valor, os colocando na mochila, resultando assim na solução ótima.

